CPU Scheduling Simulator Using C

Operating Systems Tiny Project

Student 1: Wilbard Amwele
Student 2: Ayush Baberwal
Student 2: Baldhik Bhoiya

Complete C Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

void fcfs() {
    int n, i, bt[MAX], wt[MAX], tat[MAX];
    printf("\n--- FCFS Scheduling ---\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

void sjf() {
    int n, i, j, bt[MAX], p[MAX], wt[MAX], tat[MAX], temp;

    printf("\n--- SJF Scheduling ---\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
}

void roundRobin() {
    int n, tq, i, time = 0, done;
    int bt[MAX], rt[MAX];

    printf("\n--- Round Robin Scheduling ---\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("\nExecution Order:\n");
    do {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rt[i] > 0) {
                done = 0;
                if (rt[i] > tq) {
                    time += tq;
                    rt[i] -= tq;
                    printf("P%d -> ", i + 1);
                } else {
                    time += rt[i];
                    printf("P%d ", i + 1);
                    rt[i] = 0;
                }
            }
        }
    } while (!done);

    printf("\nTotal CPU time: %d\n", time);
}

void priorityScheduling() {
    int n, i, j, bt[MAX], p[MAX], pr[MAX], wt[MAX], tat[MAX], temp;

    printf("\n--- Priority Scheduling ---\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time and Priority for P%d: ", i + 1);
        scanf("%d%d", &bt[i], &pr[i]);
        p[i] = i + 1;
    }

    /* Lower numeric priority value is treated as higher priority,
       matching the ordering used in the supplied lab manual. */
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (pr[i] > pr[j]) {
                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t\t%d\t%d\t%d\n",
               p[i], pr[i], bt[i], wt[i], tat[i]);
}

void multiLevelQueue() {
    int n, i, bt[MAX], q[MAX], p[MAX];

    printf("\n--- Multi-Level Queue Scheduling ---\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time and Queue (0=High, 1=Low) for P%d: ",
               i + 1);
        scanf("%d%d", &bt[i], &q[i]);
        p[i] = i + 1;
    }
    printf("\nExecution Order:\n");
```

```
    for (i = 0; i < n; i++)
        if (q[i] == 0) printf("P%d ", p[i]);

    for (i = 0; i < n; i++)
        if (q[i] == 1) printf("P%d ", p[i]);

    printf("\n");
}

int main() {
    int choice;

    do {
        printf("\n========================================\n");
        printf("       CPU SCHEDULING SIMULATOR\n");
        printf("========================================\n");
        printf("1. FCFS Scheduling\n");
        printf("2. SJF Scheduling\n");
        printf("3. Round Robin Scheduling\n");
        printf("4. Priority Scheduling\n");
        printf("5. Multi-Level Queue Scheduling\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: fcfs(); break;
            case 2: sjf(); break;
            case 3: roundRobin(); break;
            case 4: priorityScheduling(); break;
            case 5: multiLevelQueue(); break;
            case 6: printf("\nExiting program...\n"); break;
            default: printf("\nInvalid choice. Try again.\n");
        }
    } while (choice != 6);

    return 0;
}
```